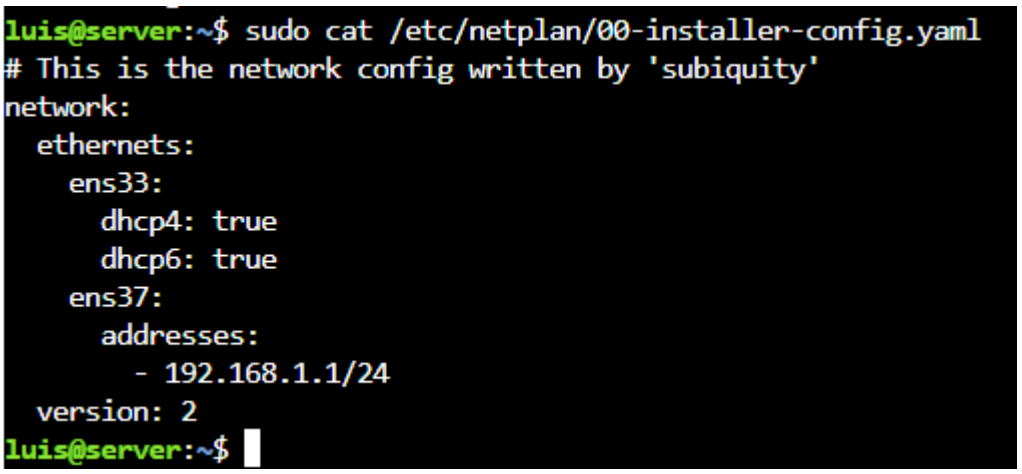
DNS - DHCP

(Su funciòn es hacer como el router de casa)

Configuración de los adaptadores del server ubuntu

sobrescribir el archivo completo desde la consola con un bloque de texto garantizado y puro.

```
# This is the network config written by 'subiquity'
network:
 ethernets:
  ens33:
   dhcp4: true
   dhcp6: true
  ens37:
   addresses:
    - 192.168.1.1/24
 version: 2
```



INSTALACION DNS BIND9

```
sudo apt update sudo apt install bind9 bind9utils -y
```

```
sudo nano /etc/bind/named.conf.options
```

```
options {
        directory "/var/cache/bind";

        // REENVIADORES: Aquí indicas a dónde viajan las peticiones de Internet
        forwarders {
                8.8.8.8;
                8.8.4.4;
        };

        // SEGURIDAD: Permite consultas desde la red local y el propio servidor
        allow-query { localhost; 192.168.1.0/24; };

        // Configuración de seguridad por defecto
        dnssec-validation auto;
        listen-on-v6 { any; };
};
```

Componente DNS: Servidor de Caché BIND 9

El servicio DNS de la infraestructura de laboratorio está gestionado por **BIND 9**, el cual se encuentra activo y escuchando en la dirección IP `192.168.1.1` a través de la interfaz de red interna `ens37` (puerto **53/UDP** y **53/TCP**).

En este escenario, el servidor no aloja zonas locales propias, sino que opera estrictamente como un **Servidor DNS de Caché Recursivo y Reenviador (Forwarder)**. Su comportamiento se define de la siguiente manera:

- **Procesamiento de consultas:** Cuando la máquina cliente Debian realiza una consulta de resolución de nombres, esta es recibida por la IP `.1`.
- **Reenvío a Internet:** Al no ser un servidor autoritativo, BIND 9 toma la petición del cliente y la delega a los reenviadores públicos de Internet configurados globalmente en el sistema, permitiendo una navegación fluida hacia los dominios externos analizados en la práctica.

INSTALACION DHCP KEA

```
sudo apt update
```

```
sudo apt install kea-dhcp4-server -y
```

sobrescribir el archivo completo desde la consola con un bloque de texto garantizado y puro.

```
sudo bash -c 'cat << EOF > /etc/kea/kea-dhcp4.conf
{
"Dhcp4": {
    "interfaces-config": {
        "interfaces": [ "ens37" ]
    },
    "lease-database": {
        "type": "memfile",
        "persist": true,
        "name": "/var/lib/kea/kea-leases4.csv"
    },
    "valid-lifetime": 4000,
    "renew-timer": 1000,
    "rebind-timer": 2000,
    "option-data": [
        {
            "name": "domain-name-servers",
            "data": "192.168.1.1"
        },
        {
            "name": "routers",
            "data": "192.168.1.1"
        }
    ],
    "subnet4": [
        {
            "id": 1,
            "subnet": "192.168.1.0/24",
            "pools": [
                {
                    "pool": "192.168.1.10-192.168.1.100"
                }
            ]
        }
    ]
}
}
EOF'
```

debemos corregir los permisos de las carpetas de Kea con dos comandos rápidos:

```
# 1. Ajustar los permisos de la carpeta de configuración a lo que pide el sistema (755)

sudo chmod 755 /etc/kea

# 2. Ajustar los permisos de la carpeta de ejecución interna de Kea

sudo chmod 755 /run/kea

# 3. Reiniciar el servicio para que lea los nuevos permisos

sudo systemctl restart kea-dhcp4-server
```

IPTABLES

`iptables` es el cortafuegos (firewall) nativo del sistema operativo Linux. Es una herramienta integrada en el núcleo (*kernel*) que se encarga de analizar, filtrar, desviar o modificar los paquetes de datos que entran, salen o atraviesan tu servidor.

Para entenderlo de forma muy sencilla en tu trabajo de redes, imagínalo como el **guardia de seguridad o el controlador de aduanas** de tu Servidor Ubuntu. Él decide qué paquetes tienen permiso para pasar y cuáles se destruyen o se transforman.

Definición Tecnológica: El Cortafuegos `iptables`

`iptables` es una herramienta de administración en espacio de usuario que permite configurar las tablas de cortafuegos, filtrado de paquetes y traducción de direcciones de red (NAT) provistas por el subsistema **Netfilter** incorporado en el núcleo de Linux.

Su arquitectura operativa se fundamenta en reglas estructuradas dentro de tablas y cadenas que evalúan de manera secuencial los encabezados de los paquetes de datos (capas 3 y 4 del modelo OSI). En este escenario de laboratorio, se interactúa específicamente con la **tabla NAT** y la cadena **POSTROUTING** mediante la acción `MASQUERADE`, delegando en el sistema operativo del Servidor Ubuntu la responsabilidad de reescribir dinámicamente los parámetros de direccionamiento IP del tráfico saliente para unificar las peticiones de la red local bajo una única interfaz con conectividad externa.

🚨 El siguiente paso crítico para tu actividad

Para que tu máquina **Cliente** (la que está en la red interna) pueda tener internet a través de este servidor, necesitas activar el **enrutamiento de paquetes** (IP Forwarding) y enmascarar el tráfico (NAT). Si no haces esto, el servidor tendrá internet, pero el cliente se quedará aislado.

Configuración de la Infraestructura de Red y Enrutamiento (Ubuntu Server)

Para que el escenario de laboratorio funcionara y la máquina cliente (Debian) pudiera conectar con el servidor de DNS seguro en Internet, fue necesario configurar el **Ubuntu Server** para que actuara como un enrutador o pasarela (*Gateway*) de la red local.

A continuación, se detallan los comandos ejecutados en el servidor y la justificación técnica de cada uno de ellos dentro de la práctica:

1. Activación del reenvío de paquetes (IP Forwarding)

Por defecto, los sistemas operativos Linux vienen bloqueados para que no reenvíen tráfico entre diferentes tarjetas de red por motivos de seguridad. Para permitir que los paquetes de la máquina Debian (conectada a la tarjeta interna `ens37`) cruzaran hacia la interfaz de Internet (`ens33`), se habilitó el reenvío en el núcleo del sistema:

```
sudo sysctl -w net.ipv4.ip_forward=1
```

Persistencia en el sistema: Para evitar que esta configuración se borrara al reiniciar el servidor, se inyectó la directiva de forma permanente dentro del archivo de configuración del sistema:

```
echo "net.ipv4.ip_forward=1" | sudo tee -a /etc/sysctl.conf
```

2. Configuración del enmascaramiento de red (NAT / Masquerade)

Como la máquina Debian utiliza una dirección IP privada interna (`192.168.1.10`), los servidores de Internet no sabrían cómo devolverle la información directamente. Se utilizó la herramienta de cortafuegos **`iptables`** para aplicar una regla de **NAT (Traducción de Direcciones de Red)**:

```
sudo iptables -t nat -A POSTROUTING -o ens33 -j MASQUERADE
```

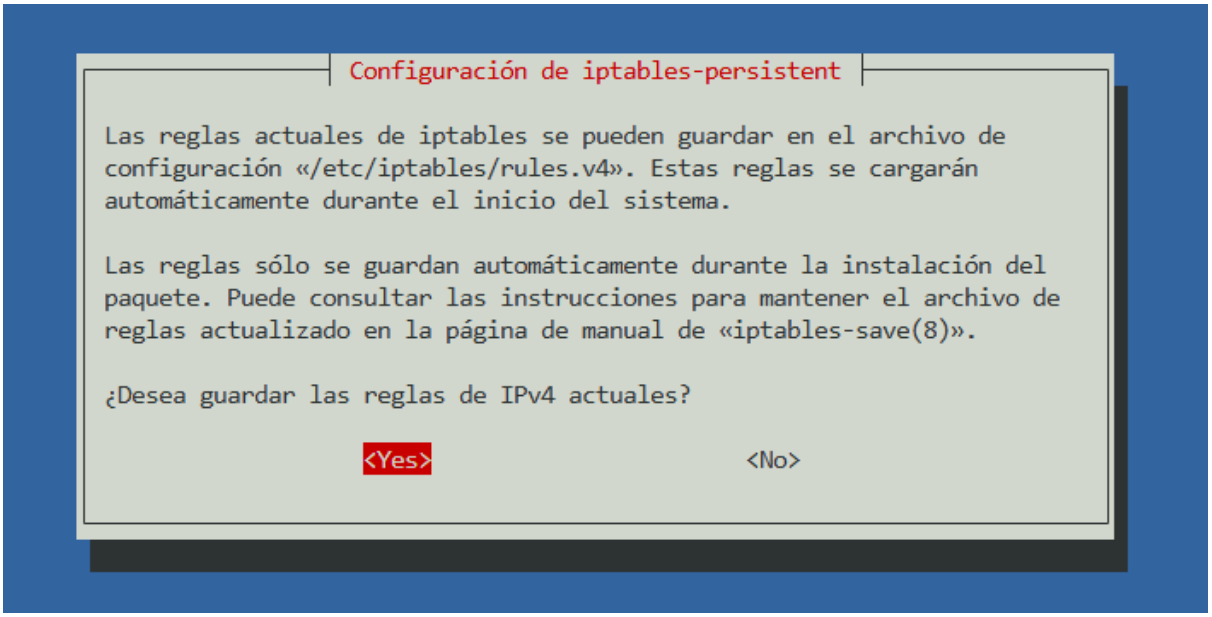
- **Significado técnico:** Esta regla le indica al servidor Ubuntu que tome cualquier paquete que provenga de la red local con destino a Internet, reemplace la IP de Debian por la IP pública o externa de su propia tarjeta `ens33`, y recuerde la conexión. Cuando Internet responde, el servidor deshace el cambio y le entrega el paquete a Debian de forma transparente.

3. Almaca instalación (La persistencia)

Poco después, al ejecutar:

```
sudo apt install iptables-persistent
```

Instalaste la herramienta para evitar esa pérdida. Durante su instalación, el asistente escaneó la memoria RAM, **detectó esa regla de `MASQUERADE` que habías puesto en el paso anterior**, y al darle tú a `<Yes>`, la copió y la guardó de forma segura en el archivo físico del disco duro (`/etc/iptables/rules.v4`).



4. Almacenamiento y persistencia de las reglas de red

Para asegurar que la infraestructura de enrutamiento se mantenga operativa de forma automática tras un apagado o mantenimiento del servidor, se salvaron las reglas de la tabla NAT en el disco duro:

```
sudo iptables-save | sudo tee /etc/iptables.rules.v4
```

5.¿Cómo probar si el servidor está haciendo su trabajo?

Para verificar que el tráfico realmente pasa por el servidor, ejecuta este comando **en la terminal de tu servidor** (mantén el comando corriendo):

```
sudo tcpdump -i ens33 icmp
```

6.Aplicar cambios:

```
sudo systemctl restart netfilter-persistent

sudo sysctl -p

cat /proc/sys/net/ipv4/ip_forward
```

Configurar el archivo resolv.conf (En tu máquina cliente Debian)

Abre el archivo de configuración DNS con permisos de administrador:

bash

```
sudo nano /etc/resolv.conf
```

Añadir los servidores DNS de Google o Cloudflare

Borra lo que tenga (o añade arriba del todo) las siguientes líneas:

text

```
nameserver 8.8.8.8

nameserver 1.1.1.1
```

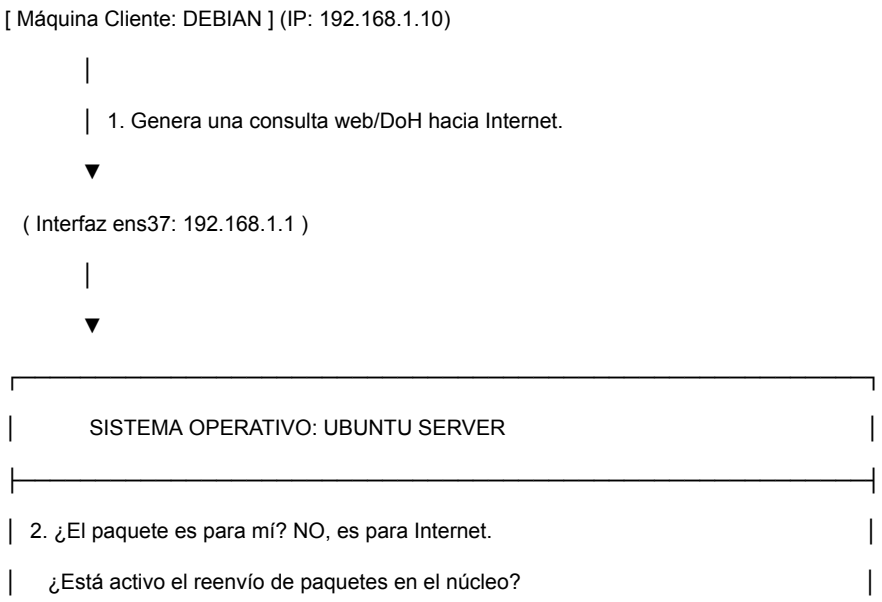
¿Cuándo se utilizan?

Estas reglas se utilizan obligatoriamente cuando necesitas transformar un servidor Linux en un enrutador para interconectar dos o más redes distintas.

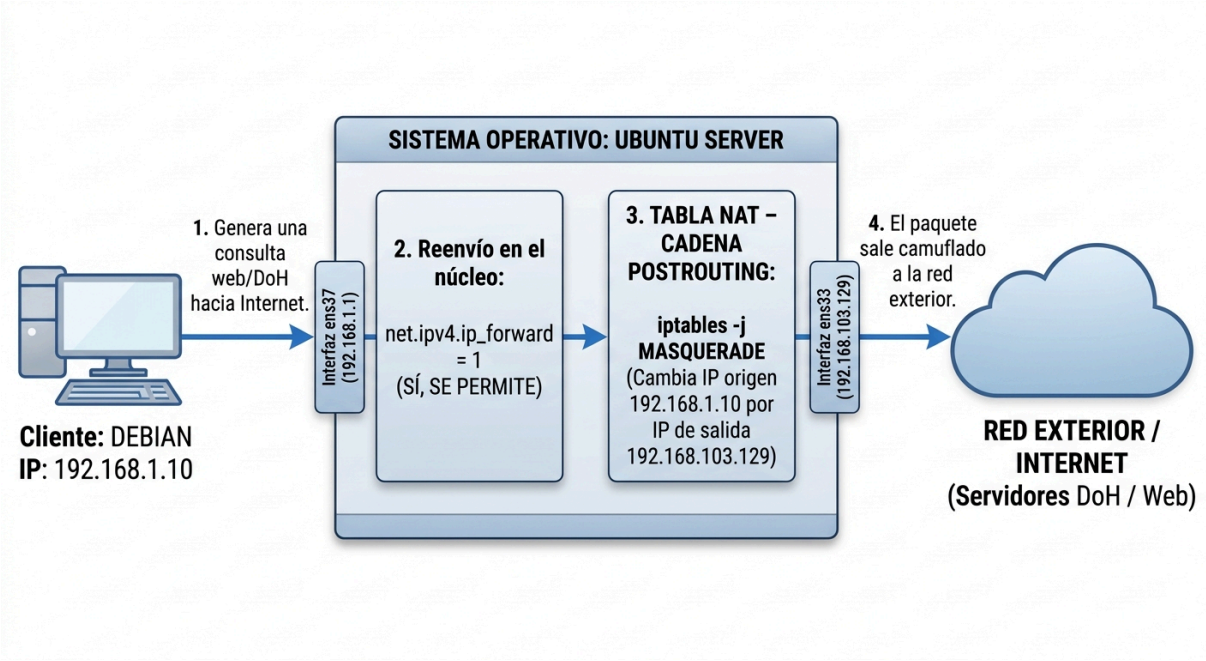
Mecanismo de Enrutamiento y NAT en Ubuntu Server

Las directivas y comandos ejecutados en el servidor corresponden a la activación del **Enrutamiento IP** y a las reglas de **Enmascaramiento NAT (Network Address Translation)**. Estos parámetros se aplican a nivel de sistema operativo en el **Servidor Ubuntu**, consolidando su rol como pasarela (*Gateway*) de la topología.

> **Ámbito del Núcleo (Kernel):** El parámetro `net.ipv4.ip_forward=1` modifica el comportamiento del núcleo de Linux de forma inmediata y permanente (vía `/etc/sysctl.conf`), autorizando al sistema operativo a conmutar y redirigir paquetes de datos entre las interfaces de red `ens37` (LAN) y `ens33` (WAN).
>
> **Ámbito del Cortafuegos (Netfilter/iptables):** La regla de la tabla NAT en la cadena de `POSTROUTING` establece un enmascaramiento dinámico (`MASQUERADE`). Su aplicación es indispensable para que el tráfico originado con direccionamiento privado en el segmento del cliente (Debian) sea traducido transparentemente con la dirección IP externa del servidor, posibilitando el establecimiento de las conexiones TLS 1.3 hacia los servidores DNS y web públicos de Internet.



```
| [ net.ipv4.ip_forward = 1 ] ➔ SÍ, SE PERMITE.                    |
└──────────────────────────┬──────────────────────────────────────┘
                           |
                           ▼ (El paquete pasa internamente)
┌──────────────────────────┴──────────────────────────────────────┐
| 3. TABLA NAT - CADENA POSTROUTING                                |
| [ iptables -j MASQUERADE ]                                       |
| • Cambia la IP de origen (192.168.1.10)                          |
|   por la IP de salida de Ubuntu (192.168.103.129).               |
| • Registra la conexión en su tabla interna.                      |
└──────────────────────────┬──────────────────────────────────────┘
                           |
                           ▼
( Interfaz ens33: 192.168.103.129 )
        |
        | 4. El paquete sale camuflado a la red exterior.
        ▼
[ RED EXTERIOR / INTERNET ] (Ej: Servidores DoH / Web)
```



La forma más segura y recomendada oficialmente por los desarrolladores de Debian y Wireshark consiste en aplicar el **Principio de Menor Privilegio**.

Tu usuario `luis` abrirá Wireshark de forma totalmente normal y segura (sin derechos de administrador), pero le daremos un permiso exclusivo del núcleo de Linux (llamado *Capability*) únicamente al motor que captura los paquetes (`dumpcap`). De esta forma, el programa puede mirar la tarjeta `ens33` pero no tiene poder para modificar nada más del sistema operativo.

Sigue estos pasos detallados, de forma limpia y ordenada:

🛠️ Paso 1: Configurar el motor de capturas (Como Root)

1. Abre la terminal en tu máquina Debian.

Inicia sesión como el administrador supremo del sistema para hacer la configuración inicial:

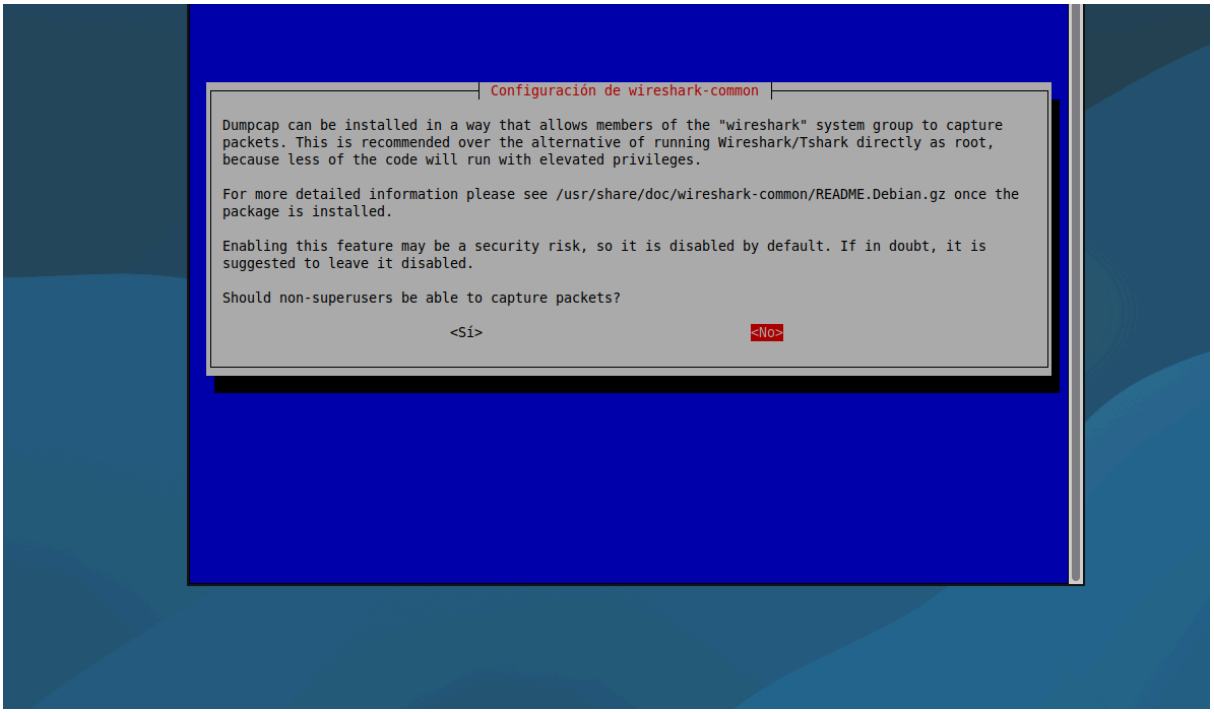
```
su -
```

Usa el código con precaución.
(Introduce tu contraseña de root y presiona Enter).

Reconfigura el paquete de Wireshark para asegurar que el sistema operativo permita a usuarios del grupo usar el motor de red:

```
dpkg-reconfigure wireshark-common
```

Te aparecerá la pantalla azul en la terminal. Asegúrate de seleccionar `<Sí>` con las flechas del teclado y pulsa ENTER.



🔑 Paso 2: Aislar los permisos del programa (Aún como Root)

Ahora ejecutaremos los comandos clave de seguridad. Le diremos a Debian que el motor `dumpcap` pertenece al grupo `wireshark` y le daremos únicamente el permiso específico de interceptar tráfico de red (`cap_net_raw,cap_net_admin`):

```
# 1. Asignar el archivo al grupo wireshark

chown root:wireshark /usr/bin/dumpcap

# 2. Permitir que solo los miembros de ese grupo puedan ejecutarlo

chmod 750 /usr/bin/dumpcap

# 3. Inyectar la "capability" de red exclusiva en el archivo (La clave de la seguridad)

setcap cap_net_raw,cap_net_admin=eip /usr/bin/dumpcap
```

👥 Paso 3: Autorizar a tu usuario y Salir

Añade a tu usuario `luis` al grupo que acabamos de autorizar para que pueda heredar ese permiso de red, y sal de la cuenta de administrador:

```
usermod -aG wireshark luis
exit
```

(Al escribir `exit`, tu terminal volverá a mostrar `luis@debian:~$`).

🔄 Paso 4: Aplicar los cambios de seguridad (Cerrar Sesión)

Linux no actualizará la lista de grupos de tu cuenta hasta que destruyas los procesos de tu sesión actual. **Este paso es obligatorio para evitar el error de "Permiso Denegado".**

1. Guarda lo que tengas abierto en tu escritorio Debian.
2. Ve a la esquina superior derecha de tu pantalla (el menú del sistema).
3. Haz clic en **Cerrar sesión** (Log out).
4. En la pantalla de bienvenida de Debian, introduce la contraseña de tu usuario `luis` para volver a entrar a tu escritorio.

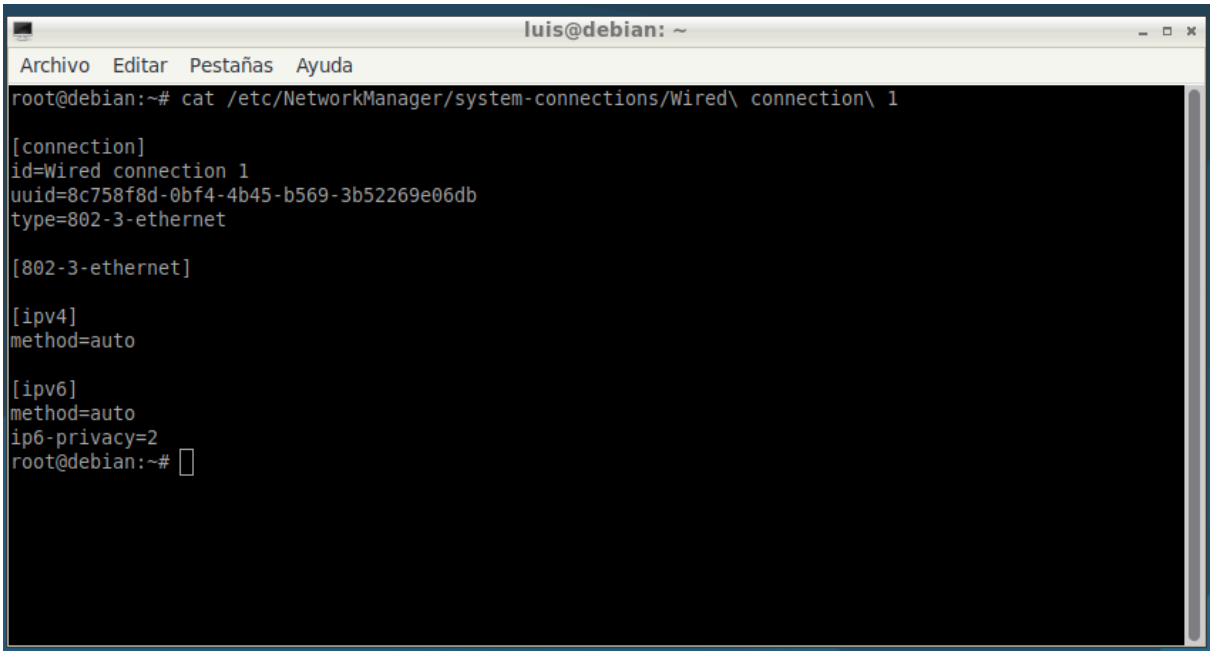
🚀 Paso 5: Lanzar Wireshark de forma 100% segura

Ahora que has vuelto a entrar a tu sesión:

1. **Abre Wireshark utilizando el ratón desde tu menú visual de aplicaciones** (o escribe simplemente `wireshark` en una terminal normal, **sin usar `sudo`, ni `su`, ni `pkexec`**).
2. Verás que el programa se abre de forma instantánea. Al no estar usando el usuario root, el entorno gráfico no protestará.
3. Gracias a la *capability* que inyectamos, **tu tarjeta `ens33` aparecerá listada perfectamente en la pantalla principal** con su gráfica de actividad.

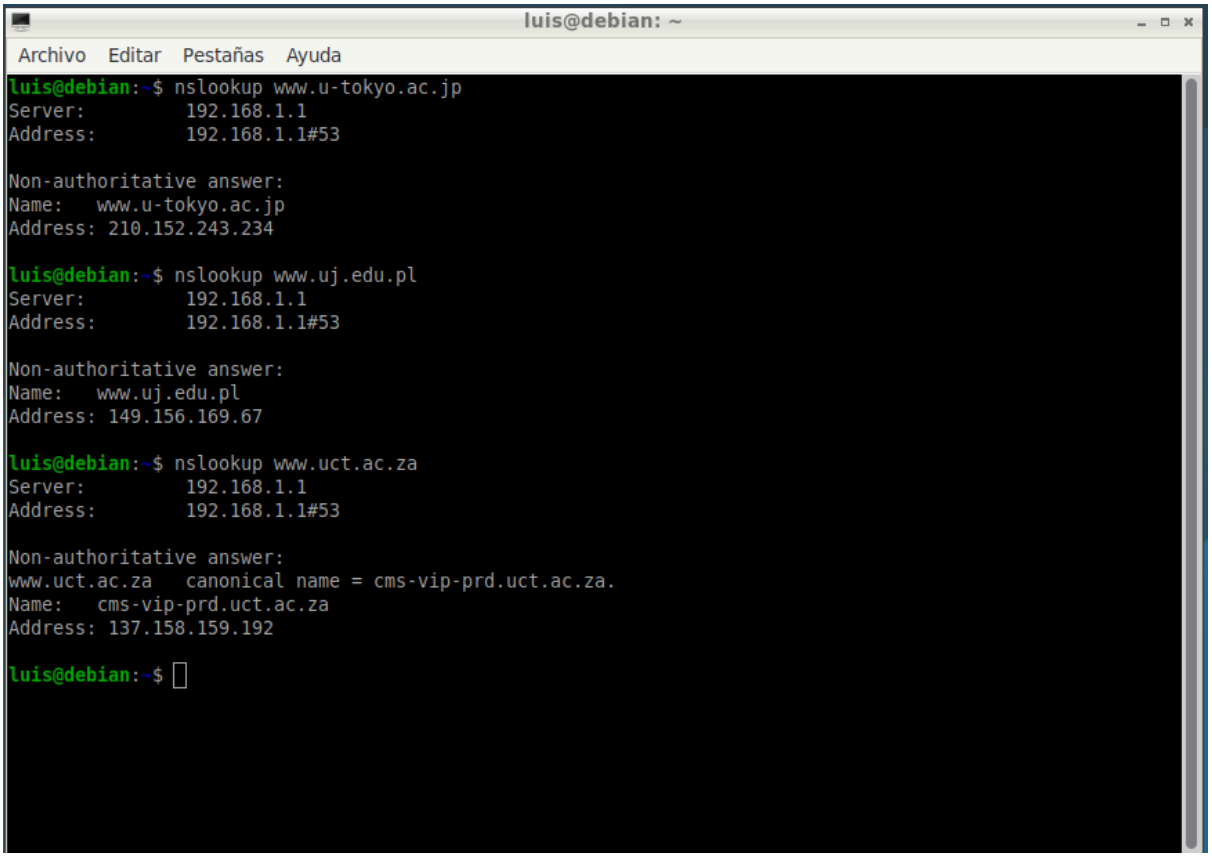
Haz **doble clic en `ens33`**, escribe **`dns`** en el filtro superior, abre una terminal normal y ejecuta tus comandos `nslookup` para registrar el tráfico de tu práctica.

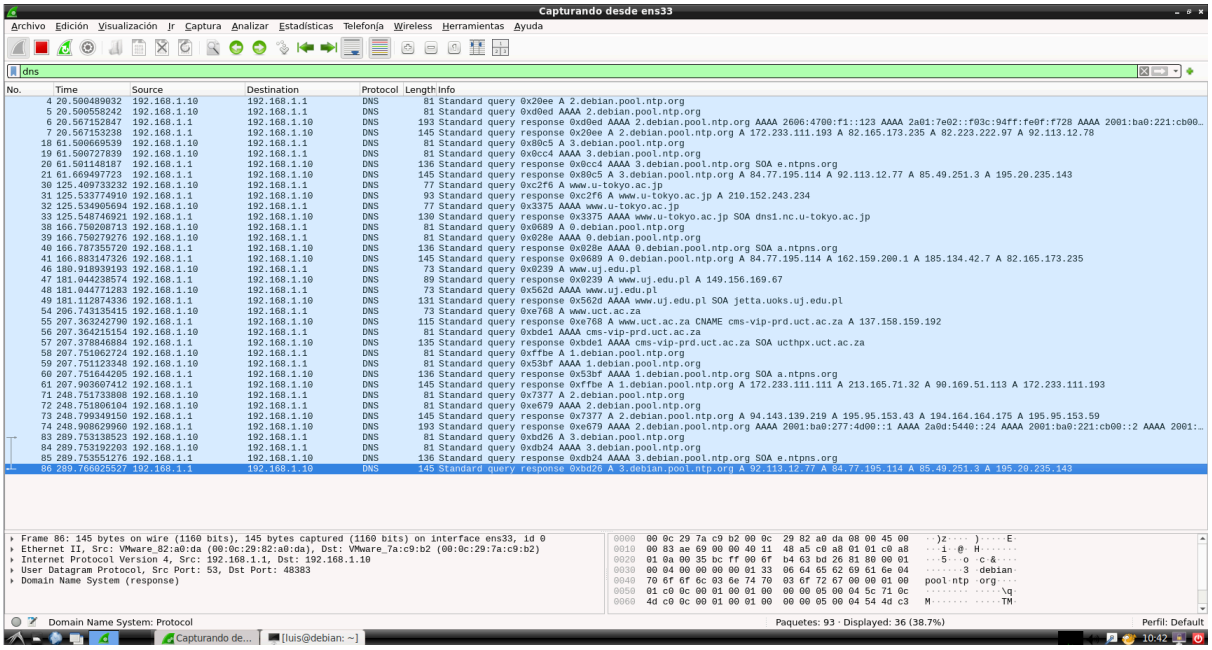
Maquina Debia (cliente) IP auto

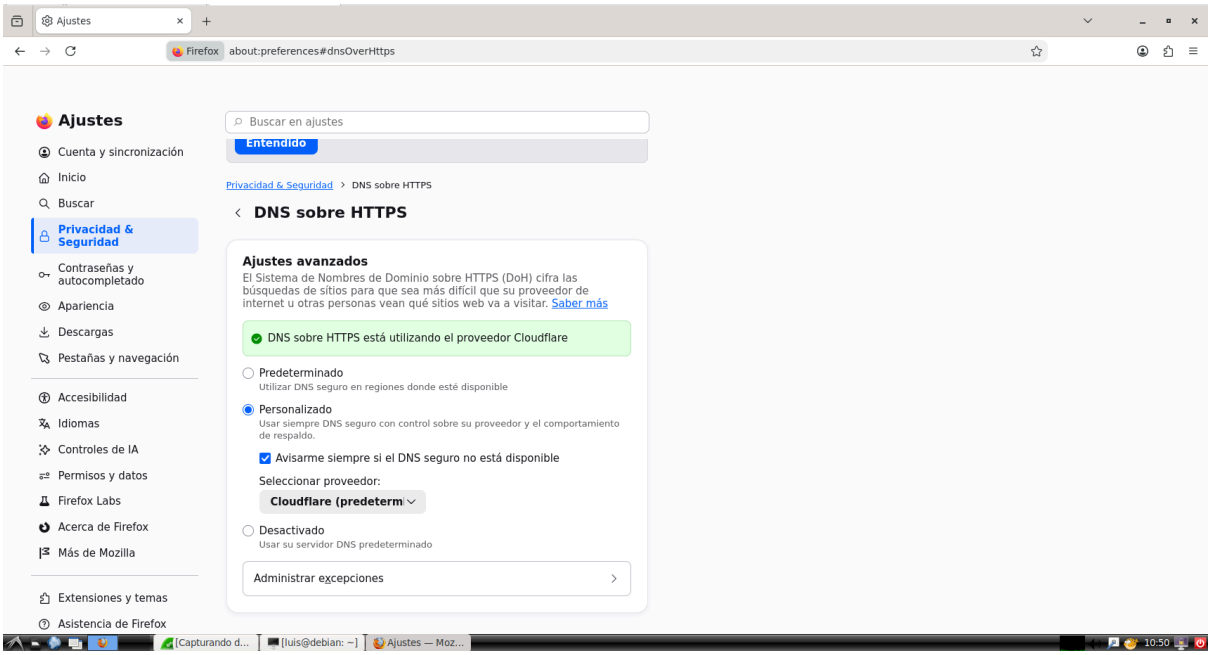


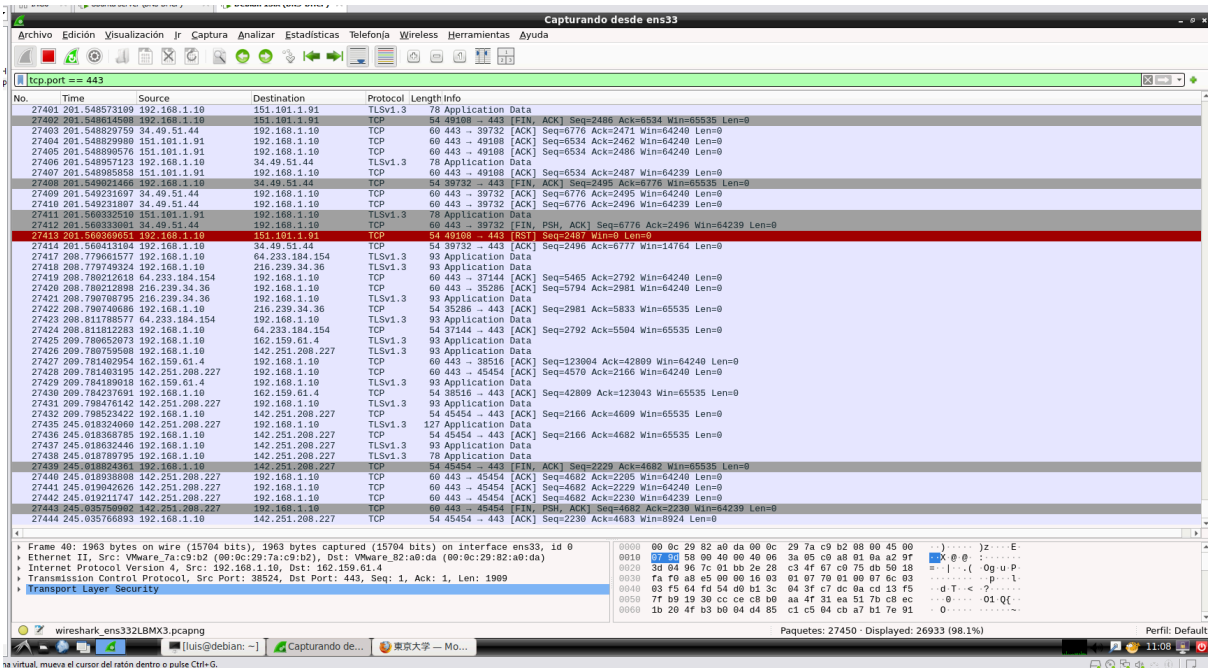
🔍 Evidencias técnicas en tu captura de pantalla:

1. **El método de IP es automático (`method=auto`):** En la sección `[ipv4]` de tu archivo de configuración `Wired connection 1`, la directiva dice claramente `method=auto`. Esto certifica que **la tarjeta de red obtiene su IP, su puerta de enlace y sus servidores DNS directamente desde tu servidor Kea DHCP (Ubuntu Server)**.


luis@debian: ~
Archivo Editar Pestañas Ayuda
luis@debian:~$ nslookup www.u-tokyo.ac.jp
Server: 192.168.1.1
Address: 192.168.1.1#53
Non-authoritative answer:
Name: www.u-tokyo.ac.jp
Address: 210.152.243.234
luis@debian:~$ nslookup www.uj.edu.pl
Server: 192.168.1.1
Address: 192.168.1.1#53
Non-authoritative answer:
Name: www.uj.edu.pl
Address: 149.156.169.67
luis@debian:~$ nslookup www.uct.ac.za
Server: 192.168.1.1
Address: 192.168.1.1#53
Non-authoritative answer:
www.uct.ac.za canonical name = cms-vip-prd.uct.ac.za.
Name: cms-vip-prd.uct.ac.za
Address: 137.158.159.192
luis@debian:~$


Capturando desde ens33
Archivo Edición Visualización Ir Captura Analizar Estadísticas Telefonía Wireless Herramientas Ayuda
dns
No. Time Source Destination Protocol Length Info
4 20.500489032 192.168.1.10 192.168.1.1 DNS 81 Standard query 0x20ee A 2.debian.pool.ntp.org
5 20.500558242 192.168.1.10 192.168.1.1 DNS 81 Standard query 0xd0ed AAAA 2.debian.pool.ntp.org
6 20.567152847 192.168.1.1 192.168.1.10 DNS 193 Standard query response 0xd0ed AAAA 2.debian.pool.ntp.org AAAA 2606:4700:f1::123 AAAA 2a01:7e02::f03c:94ff:fe0f:f728 AAAA 2001:ba0:221:cb00…
7 20.567153238 192.168.1.1 192.168.1.10 DNS 145 Standard query response 0x20ee A 2.debian.pool.ntp.org A 172.233.111.193 A 82.165.173.235 A 82.223.222.97 A 92.113.12.78
18 61.500669539 192.168.1.10 192.168.1.1 DNS 81 Standard query 0x80c5 A 3.debian.pool.ntp.org
19 61.500727839 192.168.1.10 192.168.1.1 DNS 81 Standard query 0x0cc4 AAAA 3.debian.pool.ntp.org
20 61.501148187 192.168.1.1 192.168.1.10 DNS 136 Standard query response 0x0cc4 AAAA 3.debian.pool.ntp.org SOA e.ntpns.org
21 61.669497723 192.168.1.1 192.168.1.10 DNS 145 Standard query response 0x80c5 A 3.debian.pool.ntp.org A 84.77.195.114 A 92.113.12.77 A 85.49.251.3 A 195.20.235.143
30 125.409733232 192.168.1.10 192.168.1.1 DNS 77 Standard query 0xc2f6 A www.u-tokyo.ac.jp
31 125.533774910 192.168.1.1 192.168.1.10 DNS 93 Standard query response 0xc2f6 A www.u-tokyo.ac.jp A 210.152.243.234
32 125.534905694 192.168.1.10 192.168.1.1 DNS 77 Standard query 0x3375 AAAA www.u-tokyo.ac.jp
33 125.548746921 192.168.1.1 192.168.1.10 DNS 130 Standard query response 0x3375 AAAA www.u-tokyo.ac.jp SOA dns1.nc.u-tokyo.ac.jp
38 166.750208713 192.168.1.10 192.168.1.1 DNS 81 Standard query 0x0689 A 0.debian.pool.ntp.org
39 166.750279276 192.168.1.10 192.168.1.1 DNS 81 Standard query 0x028e AAAA 0.debian.pool.ntp.org
40 166.787355720 192.168.1.1 192.168.1.10 DNS 136 Standard query response 0x028e AAAA 0.debian.pool.ntp.org SOA a.ntpns.org
41 166.883147326 192.168.1.1 192.168.1.10 DNS 145 Standard query response 0x0689 A 0.debian.pool.ntp.org A 84.77.195.114 A 162.159.200.1 A 185.134.42.7 A 82.165.173.235
46 180.918939193 192.168.1.10 192.168.1.1 DNS 73 Standard query 0x0239 A www.uj.edu.pl
47 181.044238574 192.168.1.1 192.168.1.10 DNS 89 Standard query response 0x0239 A www.uj.edu.pl A 149.156.169.67
48 181.044771283 192.168.1.10 192.168.1.1 DNS 73 Standard query 0x562d AAAA www.uj.edu.pl
49 181.112874336 192.168.1.1 192.168.1.10 DNS 131 Standard query response 0x562d AAAA www.uj.edu.pl SOA jetta.uoks.uj.edu.pl
54 206.743135415 192.168.1.10 192.168.1.1 DNS 73 Standard query 0xe768 A www.uct.ac.za
55 207.363242790 192.168.1.1 192.168.1.10 DNS 115 Standard query response 0xe768 A www.uct.ac.za CNAME cms-vip-prd.uct.ac.za A 137.158.159.192
56 207.364215154 192.168.1.10 192.168.1.1 DNS 81 Standard query 0xbde1 AAAA cms-vip-prd.uct.ac.za
57 207.378846884 192.168.1.1 192.168.1.10 DNS 135 Standard query response 0xbde1 AAAA cms-vip-prd.uct.ac.za SOA ucthpx.uct.ac.za
58 207.751062724 192.168.1.10 192.168.1.1 DNS 81 Standard query 0xffbe A 1.debian.pool.ntp.org
59 207.751123348 192.168.1.10 192.168.1.1 DNS 81 Standard query 0x53bf AAAA 1.debian.pool.ntp.org
60 207.751644205 192.168.1.1 192.168.1.10 DNS 136 Standard query response 0x53bf AAAA 1.debian.pool.ntp.org SOA a.ntpns.org
61 207.903607412 192.168.1.1 192.168.1.10 DNS 145 Standard query response 0xffbe A 1.debian.pool.ntp.org A 172.233.111.111 A 213.165.71.32 A 90.169.51.113 A 172.233.111.193
71 248.751733808 192.168.1.10 192.168.1.1 DNS 81 Standard query 0x7377 A 2.debian.pool.ntp.org
72 248.751806104 192.168.1.10 192.168.1.1 DNS 81 Standard query 0xe679 AAAA 2.debian.pool.ntp.org
73 248.799349150 192.168.1.1 192.168.1.10 DNS 145 Standard query response 0x7377 A 2.debian.pool.ntp.org A 94.143.139.219 A 195.95.153.43 A 194.164.164.175 A 195.95.153.59
74 248.908629960 192.168.1.1 192.168.1.10 DNS 193 Standard query response 0xe679 AAAA 2.debian.pool.ntp.org AAAA 2001:ba0:277:4d00::1 AAAA 2a0d:5440::24 AAAA 2001:ba0:221:cb00::2 AAAA 2001:…
83 289.753138523 192.168.1.10 192.168.1.1 DNS 81 Standard query 0xbd26 A 3.debian.pool.ntp.org
84 289.753192203 192.168.1.10 192.168.1.1 DNS 81 Standard query 0xdb24 AAAA 3.debian.pool.ntp.org
85 289.753551276 192.168.1.1 192.168.1.10 DNS 136 Standard query response 0xdb24 AAAA 3.debian.pool.ntp.org SOA e.ntpns.org
86 289.766025527 192.168.1.1 192.168.1.10 DNS 145 Standard query response 0xbd26 A 3.debian.pool.ntp.org A 92.113.12.77 A 84.77.195.114 A 85.49.251.3 A 195.20.235.143
Frame 86: 145 bytes on wire (1160 bits), 145 bytes captured (1160 bits) on interface ens33, id 0
Ethernet II, Src: VMware_82:a0:da (00:0c:29:82:a0:da), Dst: VMware_7a:c9:b2 (00:0c:29:7a:c9:b2)
Internet Protocol Version 4, Src: 192.168.1.1, Dst: 192.168.1.10
User Datagram Protocol, Src Port: 53, Dst Port: 48383
Domain Name System (response)
0000 00 0c 29 7a c9 b2 00 0c 29 82 a0 da 08 00 45 00
0010 00 83 ae 69 00 00 40 11 48 a5 c0 a8 01 01 c0 a8
0020 01 0a 00 35 bc ff 00 6f b4 63 bd 26 81 80 00 01
0030 00 04 00 00 00 00 01 33 06 64 65 62 69 61 6e 04
0040 70 6f 6f 6c 03 6e 74 70 03 6f 72 67 00 00 01 00
0050 01 c0 0c 00 01 00 01 00 00 00 05 00 04 5c 71 0c
0060 4d c0 0c 00 01 00 01 00 00 00 05 00 04 54 4d c3
Domain Name System: Protocol
Paquetes: 93 · Displayed: 36 (38.7%)
Perfil: Default
Capturando de...
[luis@debian: ~]
10:42





Introducción: ¿Qué es DNS sobre HTTPS (DoH) y por qué se utiliza?

1. El problema del DNS tradicional (Antes de activar DoH)

De forma nativa, el protocolo DNS (Sistema de Nombres de Dominio) funciona enviando consultas en **texto plano** a través del puerto **53/UDP**. Esto significa que cuando un usuario escribe una dirección web en su navegador (por ejemplo,

`www.u-tokyo.ac.jp`), la pregunta viaja por la red local e Internet sin ningún tipo de cifrado ni protección.

Este comportamiento plantea dos graves problemas de seguridad y privacidad:

- **Espionaje (Sniffing):** Cualquier atacante o administrador de red en el mismo entorno local puede interceptar el tráfico con herramientas como Wireshark y ver con absoluta precisión qué páginas web está intentando visitar el usuario.
- **Suplantación de identidad (Spoofing / MitM):** Al no estar protegida, un atacante intermedio podría alterar la respuesta del servidor DNS en el camino, enviando al usuario a una página web falsa o clonada para robar sus datos.

2. ¿Por qué se implementa DNS sobre HTTPS (DoH)?

Para solucionar de raíz estas vulnerabilidades, se desarrolló el estándar **DoH (DNS sobre HTTPS)**. Su objetivo principal es aplicar el mismo nivel de seguridad que ya utilizamos para realizar compras o transacciones bancarias en la web a las consultas de nombres de dominio.

Al activar DoH, se consiguen tres mejoras fundamentales:

- **Cifrado absoluto:** Las preguntas y respuestas DNS se encriptan utilizando el protocolo de seguridad **TLS**.
- **Camuflaje de tráfico:** Las peticiones se envían a través del puerto **443/TCP** (el puerto estándar de HTTPS). Para un observador externo o un cortafuegos, el tráfico DNS es indistinguible de la navegación web normal, impidiendo bloqueos o censuras.
- **Autenticidad:** Garantiza que la respuesta proviene de un proveedor DNS de confianza y que no ha sido modificada por terceras personas en el trayecto.

3. ¿Cómo funciona el proceso?

El flujo de comunicación cambia por completo siguiendo este orden:

1. **La Petición:** El usuario introduce la URL en el navegador Firefox de su máquina cliente (`192.168.1.10`).
2. **El Cifrado:** Firefox toma esa consulta DNS, la envuelve dentro de una conexión cifrada segura (`TLSv1.3`) y la dirige directamente hacia un proveedor público seguro (en este caso, los servidores de Cloudflare).
3. **El Tránsito:** El tráfico atraviesa la red local y cruza el servidor enrutador (Ubuntu Server) hacia el exterior a través del puerto 443.
4. **La Respuesta:** El proveedor descifra la petición en sus instalaciones, busca la IP de la página universitaria solicitada, la vuelve a cifrar en el túnel TLS y

se la devuelve al navegador de forma totalmente invisible para el resto de la red.

Ámbito de Implementación de DoH

La tecnología **DNS sobre HTTPS (DoH)** se caracteriza por implementarse directamente a nivel de aplicación (en el navegador web) y no a nivel de sistema operativo. Esto permite que el navegador aísle y proteja sus propias consultas de forma autónoma.

En la actualidad, la práctica totalidad de los navegadores basados en Chromium (como **Google Chrome, Microsoft Edge, Brave y Opera**) y **Mozilla Firefox** incorporan esta funcionalidad en sus menús de seguridad de manera nativa. Para este laboratorio se seleccionó **Mozilla Firefox** debido a su gestión estricta del protocolo, la cual fuerza el desvío de las peticiones hacia el puerto 443 de forma independiente a los parámetros de red de la máquina cliente, facilitando la auditoría limpia del tráfico cifrado en Wireshark.

Resultado Final del Escenario de Laboratorio

Gracias a esta configuración combinada, la topología del laboratorio quedó totalmente unificada y validada con éxito:

1. **Kea DHCP** asignó la configuración de red y el DNS local de forma dinámica a la máquina Debian.
2. **BIND 9** quedó disponible en la IP `.1` para resolver las consultas internas tradicionales y reenviar las peticiones externas.
3. El enrutamiento **NAT** abrió el canal hacia Internet de forma segura.
4. **Firefox** pudo establecer el túnel cifrado **DoH (TLS 1.3)** en el puerto 443 hacia Cloudflare, el cual fue capturado e identificado con éxito en la interfaz de **Wireshark**.